
Grokking Algorithms Concepts

Release 1.0.0

Ali Izadi

Oct 07, 2026

1	Problems	1
1.1	Problem 1:	1
1.1.1	I can't figure out the binary search implementation!?	1
1.2	Problem 2:	1
1.2.1	I was a little confused about logarithms!	1
1.3	Problem 3:	1
1.3.1	I had no idea to understand the concept of permuatation!	1
1.4	Problem 4:	2
1.4.1	I want to inspect call stacks in a Python program that runs a recursive factorial function!	2

CHAPTER 1

PROBLEMS

1.1 Problem 1:

1.1.1 I can't figure out the binary search implementation!?

What I tried

At first I try to implement binary search without AI help or viewing book's implementation. But this is not worked. After that I use AI to get a correct implementation but my code output was not correct! Finally I used book's implementation in page 9 to correct my different program.

1.2 Problem 2:

1.2.1 I was a little confused about logarithms!

What I tried

1. Read the book's definition for logarithms.
2. Watch some tutorials in KhanAcademy about logarithms.
3. Solve some practices in KhanAcademy about logarithms.

1.3 Problem 3:

1.3.1 I had no idea to understand the concept of permuatation!

What I tried

1. Watch one video about permutation in KhanAcademy
2. Compare this concept to factorial concept ($n!$) and the ambiguity was resolved.

1.4 Problem 4:

1.4.1 I want to inspect call stacks in a Python program that runs a recursive factorial function!

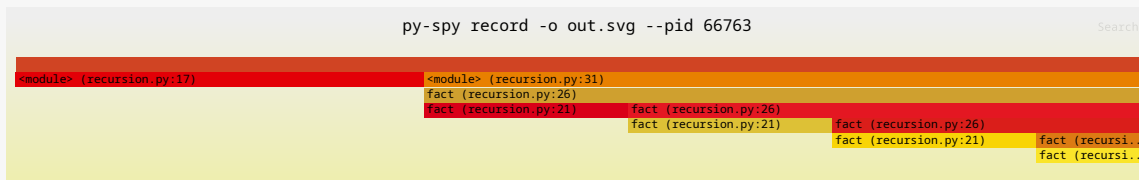
What I tried

1. I used the `strace` tool to inspect call stacks but it is not useful because `strace` just traces system calls in kernel space and does not show me user calls in user space (which Python programs use).
2. I asked AI to get help. It suggests that I use one of these tools:

- `py-spy`
- `gdb`
- `faulthandler`
- `perf`

3. First I used `py-spy`:

```
py-spy record -o out.svg --pid <PID>
```



The result is good but not perfect! because I want more details about each calling stacks.

4. After that I used `gdb` (GNU Debugger) tool:

```
gdb -p <PID>
```

But I can't use it because I did not understand some concepts in this context.

Note: If you want to run `gdb` without operation permission error you should run this command before:

```
sudo sysctl kernel.yama.pttrace_scope=0
```

5. I didn't try two other tools: `faulthandler` and `perf` for time saving.